

Introduction

A- **Installation de apache2**

1- Mettre à jour la machine
2- Renommez la machine apache2
3- Installation du paquet apache2
4- Test de l'installation
5- Comprendre l'arborescence des fichier d'apache2 Localiser les fichiers
   a- Explorer le fichier apache2.conf
   b- le contexte directory
6- Les mods d'Apache2 et leur gestion le processus
   a- Les trois mods apache2 mpm_event, mpm_worker et mpm_perfork
   b- Activation et désactivation manuelle des mods/--
      i- Activation et désactivation manuelle des mod mpm
      ii- Activation et désactivation de mods mpm avec script
      iii- Détermination du pid des processus

B- **Création d'hôte virtuelle**

1- Création du répertoire de base et de la page d'accueil index.html
2- Création d'hôte **virtuelle** par nom
3- Création d'**hôte virtuelle** par adresse IP
4- Changement de ports
5- Utilisation du PHP dans le site
   a- Implémentation du mod-PHP
   b- Implémentation du FPM PHP

C- **Sécurisation d'apache2**

1- Sécurisation apache en masquant sa version et l'os utilisé.
2- Sécurisation par SSL
3- Sécurisation des répertoires
   a. Utilisation des Virtuals host
   b. Utilisation du fichier .htaccess

Introduction

Apache est le nom de la fondation qui a devellopé le serveur *Apache httpd*, elle a développé en premier le serveur web httpd mais aussi d'autres projets à la suite.

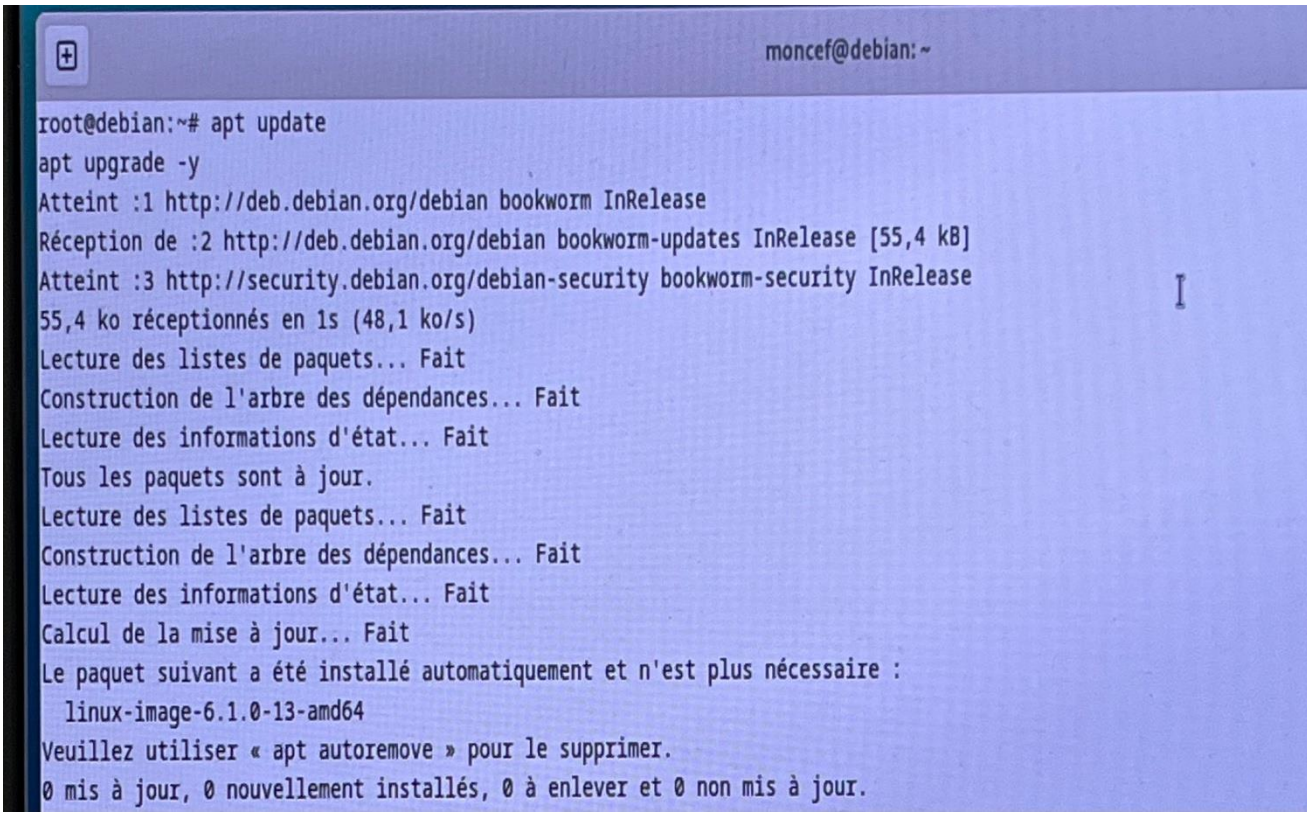
Apache existe depuis plus de 20 ans c'est le serveur web le plus utilisé au monde. Son principal concurrent est un autre logiciel libre appelé *Nginx* .

Les parts de marché sont donnée par le site suivant : https://fr.hostadvice.co:m/marketshare/

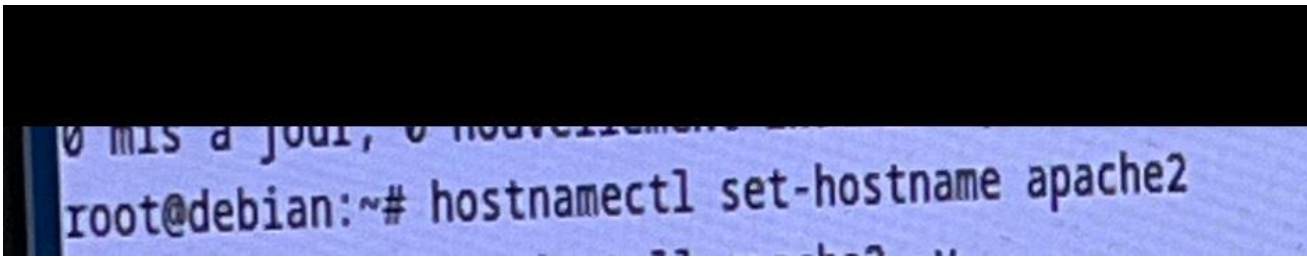
Le site officiel de apache2 est https://httpd.apache.org/

A- Installation de apache2

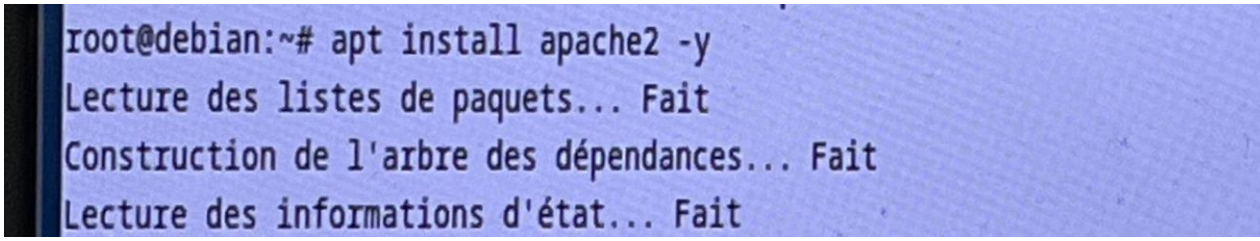
1- Mettre à jour la machine



2- Donner un nom à la ma chine apache2

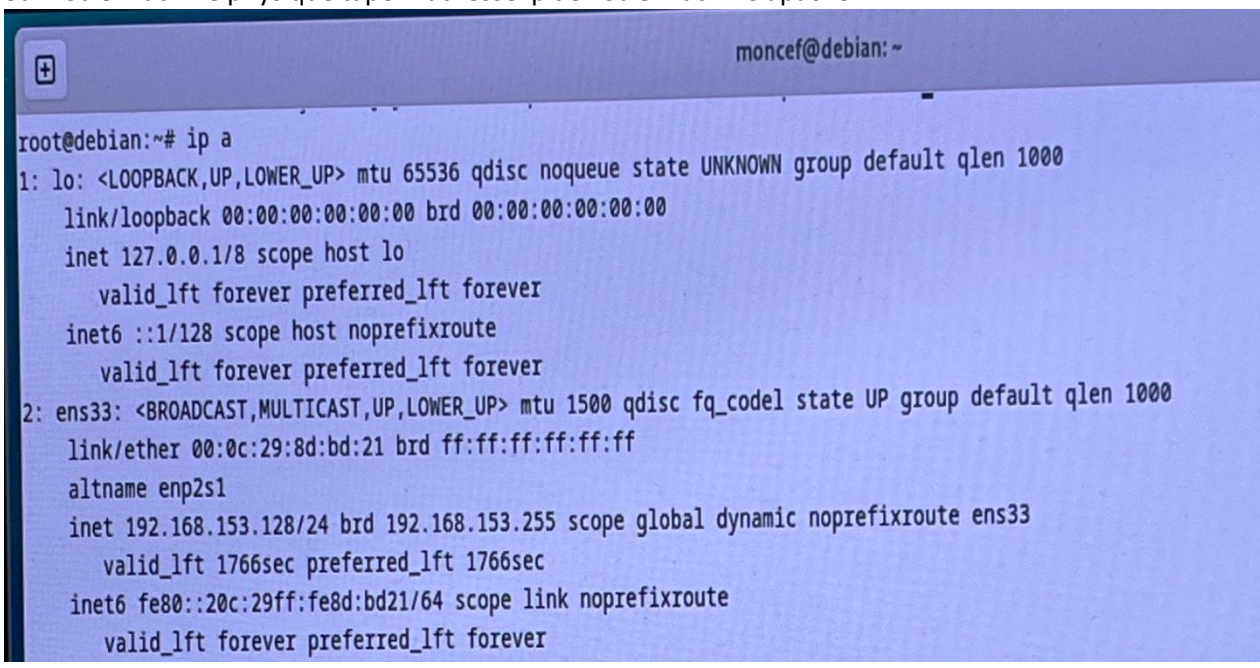


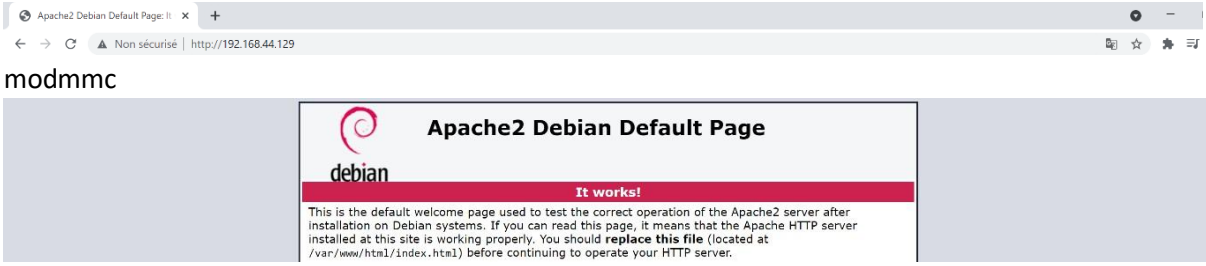
3- Installation du paquet apache2n



4- Test de la connexion apache2

Sur votre machine physique tapez l'adresse ip de votre machine apache

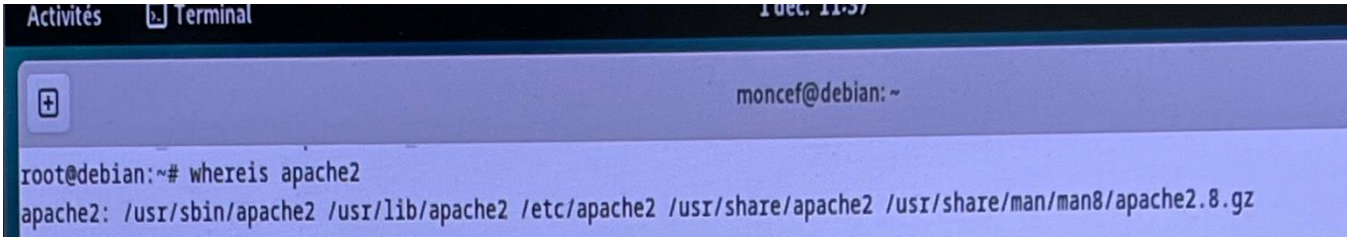




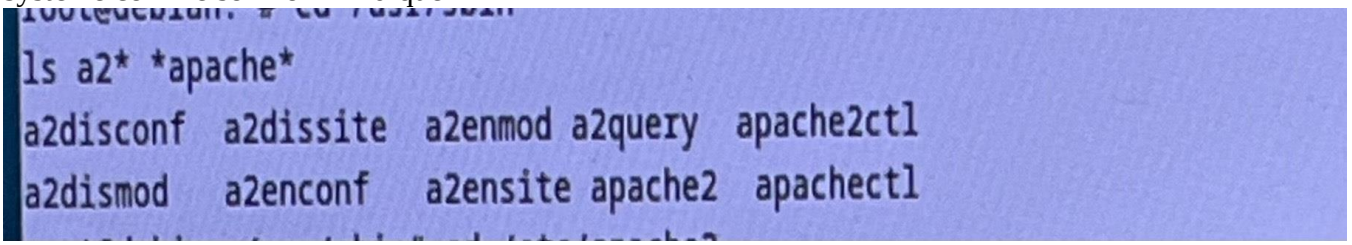
modmmc

5- Comprendre l'arborescence des fichier d'apache2 Localiser les fichieteamsrs

On va explorer les différents répertoires qui hébergent apache2, avec la commande whereis on va identifier ces répertoires.



/usr/sbin/apache2 il s'agit ici de l'exécutable dans le répertoire sbin nous avons des binaires système comme son nom l'indique

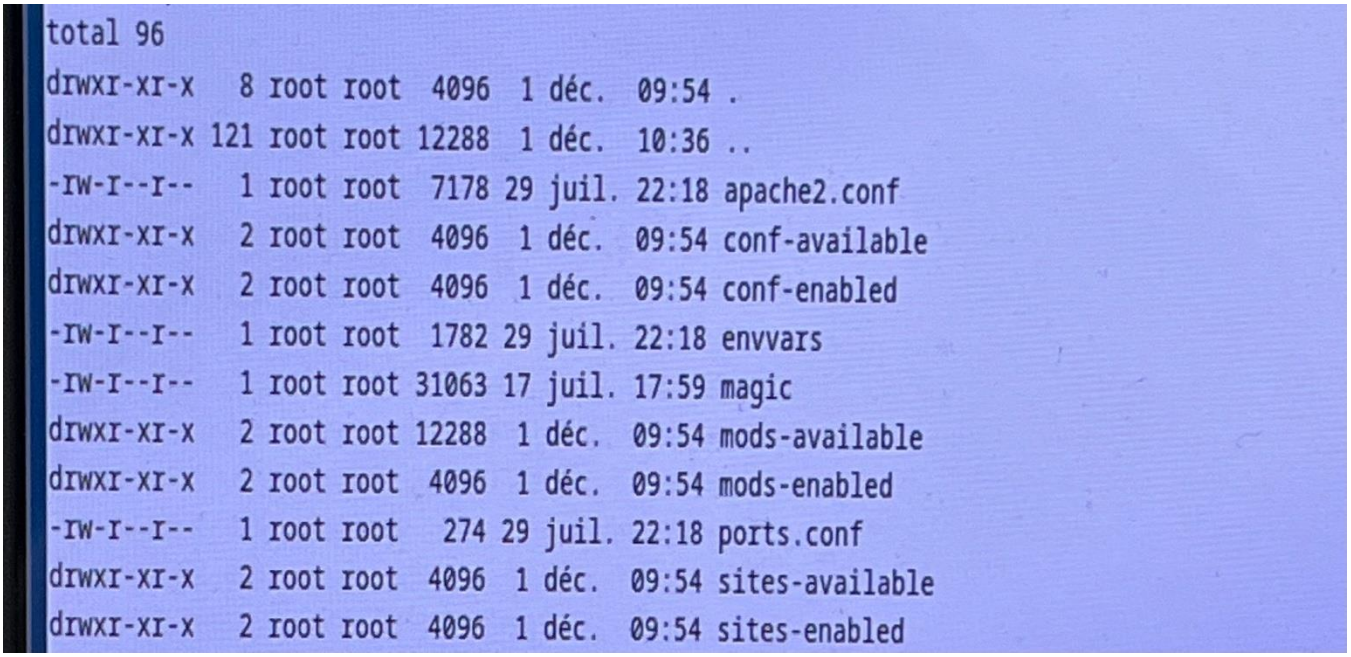


/usr/ lib/ apache2 lib pour bibliothèque
/etc/ apache2, est le répertoire où se trouve les fichiers de configuration d'apache2
/usr/ share/apache2, des fichiers partagés
/usr/share/man/man8/apache2.8.gz la documentation, obtenu avec la commande man

Le fichier **apache2.conf** nous donne une image de notre arborescence et de la configuration d'apache2.
On affiche le contenu de notre répertoire

```
cat apache2.conf
ls -al
# This is the main Apache server configuration file.  It contains the
# configuration directives that give the server its instructions.
# See http://httpd.apache.org/docs/2.4/ for detailed information about
# the directives and /usr/share/doc/apache2/README.Debian about Debian specific
# hints.
#
#
# Summary of how the Apache 2 configuration works in Debian:
# The Apache 2 web server configuration in Debian is quite different to
# upstream's suggested way to configure the web server. This is because Debian's
# default Apache2 installation attempts to make adding and removing modules,
# virtual hosts, and extra configuration directives as flexible as possible, in
# order to make automating the changes and administering the server as easy as
# possible.

# It is split into several files forming the configuration hierarchy outlined
# below, all located in the /etc/apache2/ directory:
#
# /etc/apache2/
# |-- apache2.conf
# |  `--  ports.conf
# |-- mods-enabled
# |  |-- *.load
# |  `-- *.conf
# |-- conf-enabled
# |  `-- *.conf
#  `-- sites-enabled
#   `-- *.conf
#
```



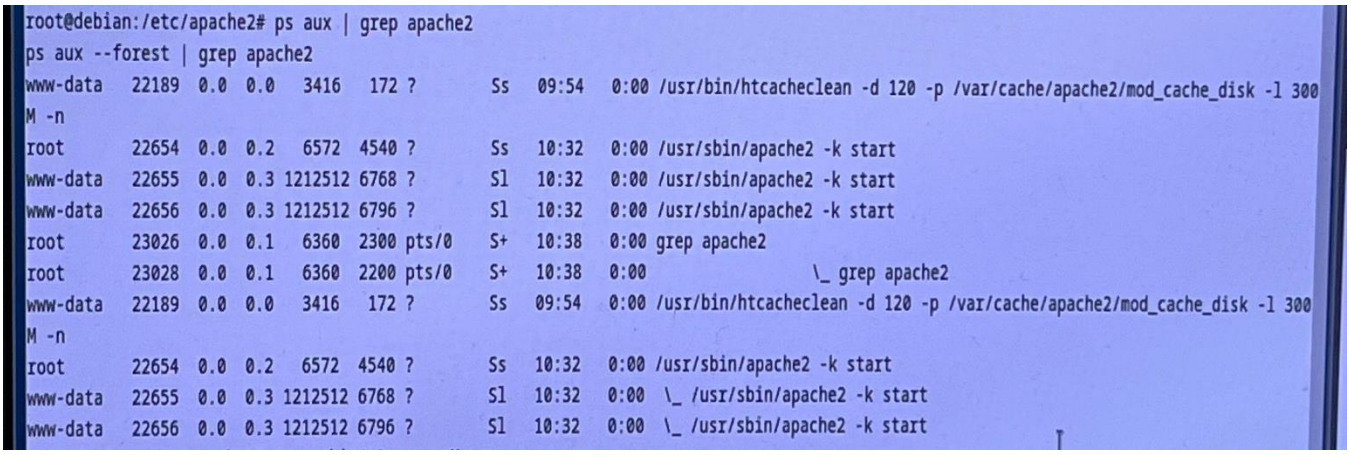
```
total 96
drwxr-xr-x   8 root root  4096  1 déc.  09:54 .
drwxr-xr-x 121 root root 12288  1 déc.  10:36 ..
-rw-r--r--   1 root root  7178 29 juil. 22:18 apache2.conf
drwxr-xr-x   2 root root  4096  1 déc.  09:54 conf-available
drwxr-xr-x   2 root root  4096  1 déc.  09:54 conf-enabled
-rw-r--r--   1 root root  1782 29 juil. 22:18 envvars
-rw-r--r--   1 root root 31063 17 juil. 17:59 magic
drwxr-xr-x   2 root root 12288  1 déc.  09:54 mods-available
drwxr-xr-x   2 root root  4096  1 déc.  09:54 mods-enabled
-rw-r--r--   1 root root   274 29 juil. 22:18 ports.conf
drwxr-xr-x   2 root root  4096  1 déc.  09:54 sites-available
drwxr-xr-x   2 root root  4096  1 déc.  09:54 sites-enabled
```

6- Les processus d'Apache et leur gestions le processus

a- Les trois mods apache2 mpm_event, mpm_worker et mpm_perfork

On utilise la commande **ps aux | grep apach[e]** pour afficher les processus d'apache2
a (all user) pour afficher les processus de tout le monde
u (user) pour montrer le propriétaire du processus
x montrer les processus qui ne sont pas attaché à un terminal

Les trois **modules MPM** (event, worker, prefork) déterminent comment Apache gère les connexions.
Il ne peut y avoir qu'un seul module MPM actif à la fois.
Le modules MPM "**event**" est démarré par défaut à l'installation d'apache2.



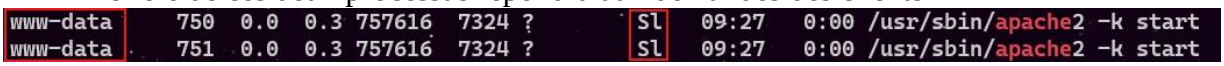
En activant soit le module event ou worker on remarque qu'il y'a 3 processus :

- Le premier processus qui tourne avec le compte root :



S = (interruptible sleep) le processus est en exécution et en attente d'un événement Il y a plusieurs threads à l'intérieur de ce processus psmaître
s = indique que le processus est master

- Les deux autres processus sont dans le même état mais avec un statut qui est en l = **(multi-threaded**) ils tournent avec un compte www-data pour des raisons de sécurité, le rôle de ces deux processus répondra aux demandes des clients



En lançant le programme plusieurs fois, des processus enfant vont répondre aux appels http des clients en même temps chaque processus a plusieurs threads vont gérer les appels des clients
Apache va charger les nouveaux processus de façon à répondre à une demande qui augmente en même temps chaque processus aura plusieurs threads afin de répondre aux demandes client.

- Si on a un **seul processu**s comportant beaucoup de threads ça peut être mal géré.
- Si on a des processus qui sont **single-threaded** on doit en ouvrir d'autres ce qui peut causer une consommation excessive de mémoire Donc apache utilise les deux méthodes, il s'agit d'une **version hybride** qui essaie de gérer au mieux une demande croissante ou fluctuante de pages web.

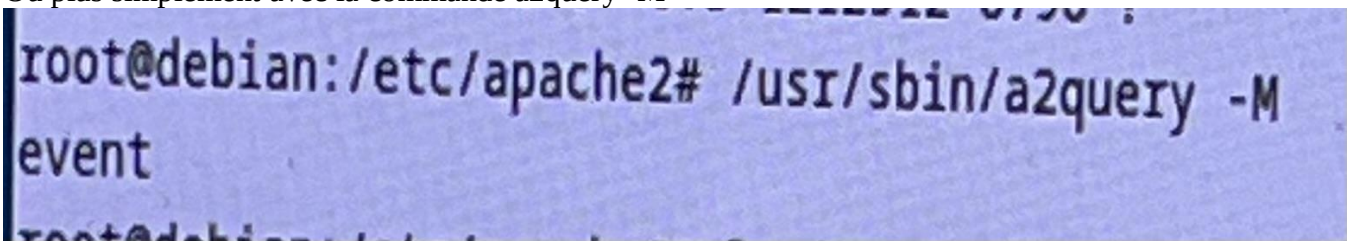
Au regard ce qui a été dit au-dessus apache peut utiliser 3 mods :

- **Worker** : Ce module permet à chaque processus fils d'Apache de gérer plusieurs connexions dans des "*threads*". Ce mode est performant mais avec des applications web qui ne gèrent pas bien ces threads il peut se montrer moins efficace.
- **Event** : c'est le module le plus récent et le mieux optimisé. C'est une amélioration de mpm_worker qui permet de mieux gérer les connexions **keepalive** (voir apache.conf) qui permettent de faire plusieurs requêtes HTTP à travers une même connexion.
  L'avantage de **mpm_events** la version améliorée de **mpm_worker** c'est sa gestion du keepalive.
  Un thread spécifique à la gestion du keepalive va permettre de diminuer le nombre de threads qui vont rester en keepalive c'est un des avantages de mpm_events qui permet une optimisation en termes
    D 'occupation de process
    D'occupation mémoire

- **Prefork** : Ce module crée un processus fils pour chaque nouvelle connexion, il est fiable mais il est gourmand en mémoire donc en termes de performance ce n'est pas l'Ideal.
  Pour vérifier quel mode est chargé pour gérer le connexion clients on va dans **/etc/apache2/mods-enabled,** on constate que le mod mpm_event est démarrée par défaut

```
root@debian:/etc/apache2/mods-enabled# ls mods-enabled/
ls: impossible d'accéder à 'mods-enabled/': Aucun fichier ou dossier de ce type
root@debian:/etc/apache2/mods-enabled# cd /etc/apache2/mods-enabled
root@debian:/etc/apache2/mods-enabled# ls
access_compat.load  authn_core.load  authz_user.load  deflate.load  filter.load      mpm_prefork.load  reqtimeout.load  status.load
alias.conf     authn_file.load  autoindex.conf   dir.conf     mime.conf       negotiation.conf  setenvif.conf
alias.load     authz_core.load  autoindex.load   dir.load     mime.load       negotiation.load  setenvif.load
auth_basic.load     authz_host.load  deflate.conf     env.load     mpm_prefork.conf  reqtimeout.conf    status.conf
```

Ou plus simplement avec la commande a2query -M



Pour arrêter un mode et en démarrer un autre on utilise les scripts créer pour ce but ou manuellement en utilisant les liens symboliques, dans cette démonstration je vais arrêter le module mpm_event et démarrer le module mpm_perfork manuellement après j'utilise les scripts crées à cet effet ce qui peut s'avérer plus simple et pratique.